Федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
Физико-механический институт
Высшая школа прикладной математики и вычислительной физики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

по курсу «Введение в технологии суперкомпьютерных вычислений»
на тему: «Распараллеливание вложенных циклов на примере вычислительных операций с трехмерной декартовой расчетной сеткой»

Работу выполнила:
Студент гр. 5030301/40102
Набережных Виктория Александровна
Преподаватели:
доцент ВШПМиВФ, к.т.н. Абрамов Алексей Геннадьевич
студент магистратуры ВШПМиВФ Мовсисян Максим Артурович

Санкт-Петербург
2026

Оглавление

1. Цель работы и постановка задачи 3
2. Информация об аппаратном и программном обеспечении 4
3. Исследование оптимизации компилятора и порядка вложенности циклов 4
4. Формулировка параллельного алгоритма для соответствующей операции 6
5. Исходные коды разработанных программ с комментариями к распараллеленным областям и используемым конструкциям OpenMP 7
6. Результаты проверки правильности выполнения программы и визуализация расчетной сетки 8
7. Исследование эффективности распараллеливания вложенных циклов 9
8. Исследование масштабируемости и сопоставительное сравнение параллельных подходов 13
Выводы 14
Приложение (код программы) 15

1. Цель работы и постановка задачи

Целью данной работы является изучение возможностей технологии OpenMP применительно к распараллеливанию вложенных циклов, в которых производятся независимые операции с многомерными массивами, на примере вычислительных операций с трехмерной декартовой расчетной сеткой.

Для достижения поставленной цели необходимо разработать распараллеленную средствами OpenMP программу на языке Fortran, выполняющую вычисление расстояний между геометрическими центрами граней ячеек заданной трехмерной структурированной декартовой расчетной сетки, и провести комплексное исследование эффективности ее выполнения при различных параметрах компиляции и запуска.

Математическая постановка задачи: задана трехмерная декартова расчетная сетка, содержащая $N_x \times N_y \times N_z$ узлов с индексами $i \in [1, N_x]$, $j \in [1, N_y]$, $k \in [1, N_z]$ и координатами узлов $(x_{i,j,k}, y_{i,j,k}, z_{i,j,k})$. Для каждой ячейки (i, j, k), где $i \in [1, N_x - 1]$, $j \in [1, N_y - 1]$, $k \in [1, N_z - 1]$, шаги сетки по координатным направлениям определяются как:

$$\Delta x_i = x_{i+1,j,k} - x_{i,j,k}\ ;\ \Delta y_j = y_{i,j+1,k} - y_{i,j,k}\ ;\ \Delta z_k = z_{i,j,k+1} - z_{i,j,k}$$

Требуется вычислить два набора геометрических характеристик для каждой ячейки сетки:

1. **Расстояния между противоположными гранями ячейки (i, j, k) вдоль осей X, Y, Z:**

$$d^{(x)}_{i,j,k} = x_{i+1,j,k} - x_{i,j,k}\ ;\ d^{(y)}_{i,j,k} = y_{i,j+1,k} - y_{i,j,k}\ ;\ d^{(z)}_{i,j,k} = z_{i,j,k+1} - z_{i,j,k}$$

2. **Расстояния между геометрическими центрами перпендикулярных граней одной ячейки:**

$$d^{(xy)}_{i,j,k} = \frac{1}{2}\sqrt{\Delta x_i^2 + \Delta y_j^2}\ ;\ d^{(xz)}_{i,j,k} = \frac{1}{2}\sqrt{\Delta x_i^2 + \Delta z_k^2}\ ;\ d^{(yz)}_{i,j,k} = \frac{1}{2}\sqrt{\Delta y_j^2 + \Delta z_k^2}$$

В программе, написанной на языке Fortran, рассматривается кубическая сетка размером $N \times N \times N$ узлов индексации от 1 до N, число ячеек $M = N - 1$ по каждому направлению. Для получения наглядной картины распределения геометрических характеристик при визуализации сетка инициализируется с легким нелинейным сгущением узлов:

$$x_{i,j,k} = \left(\frac{i-1}{N-1}\right)^{1{,}15};\ y_{i,j,k} = \left(\frac{j-1}{N-1}\right)^{1{,}10};\ z_{i,j,k} = \left(\frac{k-1}{N-1}\right)^{1{,}05}$$

Чтобы операции с памятью и вычисления занимали заметное процессорное время (порядка 0,1– 0,4 секунды) и накладные расходы на управление нитями не перекрывали полезную работу, в вычислительных экспериментах выбран размер расчетной сетки $N = 300$ ($300 \times 300 \times 300 = 2{,}7 \cdot 10^7$ узлов, или $299^3 \approx 2{,}67 \cdot 10^7$ ячеек). Для построения наглядных графиков самой сетки и полей расстояний использовалась сетка размером $N = 25$.

Для анализа производительности и масштабируемости программы введены следующие метрики:

1. Время выполнения Tp измеряется с помощью функции реального (астрономического) времени omp_get_wtime(), которая возвращает время в секундах (используется тип real(8)).

2. Ускорение Sp на p потоках по отношению к последовательной версии T1:

$$Sp = \frac{T_1}{T_p}$$

3. Эффективность распараллеливания Ep:

$$E_p = \frac{S_p}{P} = \frac{T_1}{p \cdot T_p}$$

В идеальном случае Sp ≈ p, а Ep ≈ 1,0 (100%).

2. Информация об аппаратном и программном обеспечении

Для проведения вычислительных экспериментов и замера времени работы программы использовался ноутбук на базе операционной системы macOS. Написание и редактирование исходного кода выполнялось в текстовом редакторе Geany, а автоматизация серии запусков с различными параметрами с помощью командного скрипта Bash (run3.sh).

Аппаратная конфигурация компьютера:

- *Процессор*: Apple M1 (архитектура ARM64, 8 ядер: 4 производительных ядра и 4 энергоэффективных ядра)
- *Оперативная память*: 16 ГБ единой памяти
- *Максимальное число потоков*: в среде OpenMP процессор определяется как 8 поточный

Использованное программное обеспечение:

- *Компилятор*: gfortran (GNU Fortran, установлен через пакетный менеджер Homebrew)
- *Ключи компиляции*: флаг -fopenmp для поддержки директив многопоточности OpenMP, а также флаги -O0 (отключение оптимизации для режима Debug) и -O2 (включение оптимизации компилятора для режима Release) в соответствии с программой испытаний
- *Инструменты анализа результатов*: среда Jupyter Notebook, язык Python 3 с библиотеками pandas (обработка таблиц с замерами) и matplotlib (визуализация расчетной сетки, полей расстояний и построение графиков)

3. Исследование оптимизации компилятора и порядка вложенности циклов

Было исследовано, как на время работы последовательной версии программы влияют: уровень оптимизации компилятора gfortran (-O0 и -O2) и порядок вложенности циклов при обходе трехмерных массивов.

Эксперимент проводился на одном вычислительном потоке (OMP_NUM_THREADS=1) для расчетной сетки размером N = 300 ($300 \times 300 \times 300 = 2{,}7 \cdot 10^7$ узлов, или $299^3 \approx 2{,}67 \cdot 10^7$ ячеек). Программа была скомпилирована в двух режимах:

Режим без оптимизации (флаг -O0):

bash
```
gfortran -O0 -fopenmp lab3.f90 -o lab3_O0
```

Режим с оптимизацией (флаг -O2):

bash
```
gfortran -O2 -fopenmp lab3.f90 -o lab3_O2
```

Для каждого режима компиляции были протестированы три варианта порядка вложенности циклов:

SERIAL (kji) внешний цикл идет по индексу k, средний по j, внутренний по первому индексу i
Serial jki (jki) смешанный порядок, где внутренним также остается индекс i
Serial ijk (ijk) обратный порядок, где во внутреннем цикле меняется последний индекс k

Полученные результаты приведены в таблице 1 и на рисунке 2.

Порядок циклов	**Без оптимизации (-O0)**	**С оптимизацией (-O2)**	**Ускорение от -O2**
SERIAL (kji)	0,384653	0,142583	в 2,70 раза
Serial jki (jki)	0,359589	0,244811	в 1,47 раза
Serial ijk (ijk)	2,373488	1,304945	в 1,82 раза

Таблица 1. Влияние порядка вложенности циклов и опций оптимизации на время работы (сек)

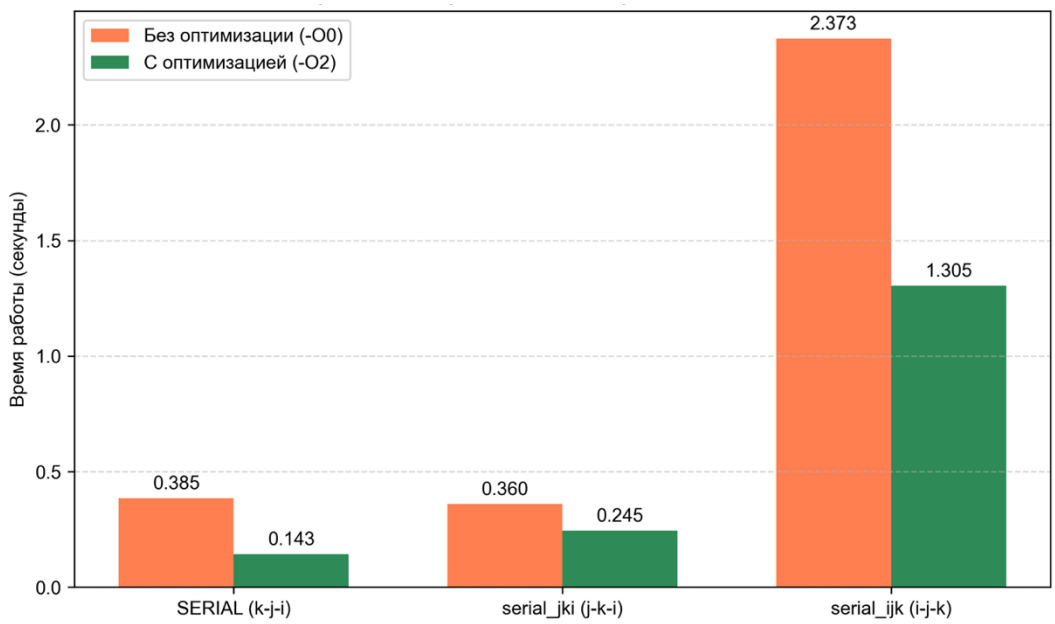


Рисунок 2. Влияние порядка вложенности циклов и оптимизации компилятора

В Fortran массивы хранятся в памяти по столбцам, поэтому соседними в памяти являются элементы, у которых меняется первый индекс i.

В варианте SERIAL внутренний цикл идет по i, поэтому процессор читает данные подряд и хорошо попадает в кэш. Время работы при -O2 составило 0,143 секунды.

В варианте serial ijk во внутреннем цикле меняется k. На каждом шаге процессор перепрыгивает через большие участки памяти и промахивается мимо кэша, из-за чего время выросло в 9 раз до 1,305 секунды.

Промежуточный вариант serial jki сохраняет быстрый внутренний цикл по i, но из-за перестановки внешних слоев уступает эталонному SERIAL (0,245 секунды).

Включение флага -O2 ускорило оптимальный алгоритм SERIAL в 2,7 раза (с 0,385 до 0,143 секунды). Компилятор применил векторные инструкции процессора и начал считать сразу несколько квадратных корней за один раз.

Весь расчет шел на одном ядре, поэтому для дальнейшего ускорения нужно использовать OpenMP.

4. Формулировка параллельного алгоритма для соответствующей операции

В данной работе выполняется геометрическая обработка трехмерной декартовой расчетной сетки размером $N \times N \times N$ узлов индексации от 1 до N, число ячеек $M = N - 1$ по каждому направлению. Для каждой ячейки (i, j, k) рассчитываются три расстояния между противоположными гранями $d^{(x)}$, $d^{(y)}$, $d^{(z)}$ и три расстояния между геометрическими центрами смежных граней $d^{(xy)}$, $d^{(xz)}$, $d^{(yz)}$.

Реализованы два разных параллельных OpenMP подхода:

1. ***Распараллеливание вложенных циклов (!$omp parallel do)***

Тройной цикл обхода сетки имеет порядок kji. Директива !$omp do ставится перед внешним циклом по индексу k (вариант omp_outer). Планировщик OpenMP делит слои сетки вдоль оси z на непрерывные блоки и раздает их потокам, благодаря чему каждая нить работает со своим сплошным куском оперативной памяти и реже промахивается мимо кэша.

Внутри своего блока слоев каждый поток перебирает ячейки по индексам j и i, находит шаги сетки lx, ly, lz в свои private переменные и записывает готовые расстояния напрямую в свои ячейки shared выходных массивов.

Для исследования накладных расходов директива также переносилась на средний (omp_mid) и внутренний (omp_inner) циклы, а для изучения объединения циклов применялась опция collapse(m) при m = 1, 2, 3.

2. ***Распараллеливание по независимым секциям (!$omp parallel sections)***

Здесь вместо нарезки массива по ячейкам вся задача делится на три независимые секции (!$omp section), каждая из которых отвечает за вычисления вдоль своего координатного направления:

1. Первая вычисляет расстояния вдоль оси X ($d^{(x)}$) и между центрами смежных граней в плоскости XY ($d^{(xy)}$)
2. Вторая вычисляет расстояния вдоль оси Y ($d^{(y)}$) и в плоскости YZ ($d^{(yz)}$)
3. Третья вычисляет расстояния вдоль оси Z ($d^{(z)}$) и в плоскости XZ ($d^{(xz)}$)

Все три секции запускаются одновременно на доступных ядрах процессора. В конце параллельной области потоки дожидаются друг друга, после чего все шесть выходных массивов оказываются полностью заполнены без гонки данных.

№	Вариант	Суть подхода
1	SERIAL	Последовательный расчет (k j i) на одном ядре без директив OpenMP
2	serial_jki / serial_ijk	Последовательные версии с измененным порядком циклов для проверки работы кэша
3	omp_outer	Основной параллельный цикл !$omp do перед внешним циклом k (непрерывные блоки памяти) с поддержкой schedule(runtime)
4	omp_mid / omp_inner	Перенос директивы !$omp do на средний j и внутренний i цикл для оценки накладных расходов на синхронизацию
5	collapse2 / collapse3	Объединение двух (k, j) и трех (k, j, i) вложенных циклов в один с помощью атрибута collapse(m)
6	SECTIONS	Разделение вычислений на 3 независимые параллельные секции !$omp sections по направлениям (X, Y, Z)

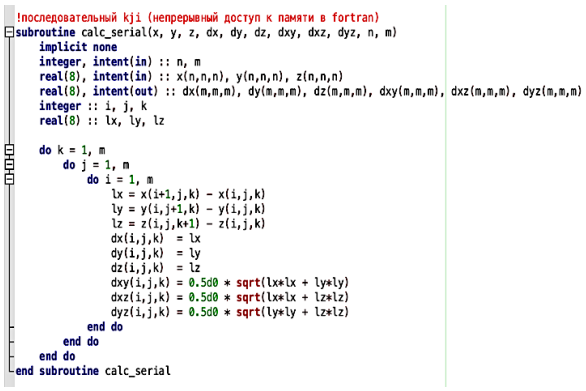
5. Исходные коды разработанных программ с комментариями к распараллеленным областям и используемым конструкциям OpenMP

Полный код программы представлен в приложении. В этом пугкте представлены основные расчетные подпрограммы, реализующие последовательный алгоритм и параллельные модификации с использованием директив OpenMP.

Во всех параллельных версиях соблюдены следующие принципы работы с памятью:

- Private изолирует счетчики циклов (i, j, k) и промежуточные длины шагов (lx, ly, lz) внутри каждого потока, чтобы нити не перезаписывали данные друг друга.

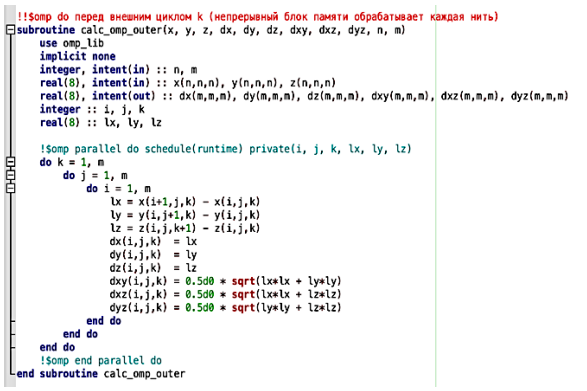
- Общие трехмерные массивы узлов сетки (x, y, z) и выходных расстояний (dx, dy, dz, dxy, dxz, dyz) разделяются между потоками без конфликтов записи, так как каждая нить пишет строго в свой набор ячеек (i, j, k).



```
!последовательный kji (непрерывный доступ к памяти в fortran)
subroutine calc_serial(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
    implicit none
    integer, intent(in) :: n, m
    real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
    real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    integer :: i, j, k
    real(8) :: lx, ly, lz

    do k = 1, m
        do j = 1, m
            do i = 1, m
                lx = x(i+1,j,k) - x(i,j,k)
                ly = y(i,j+1,k) - y(i,j,k)
                lz = z(i,j,k+1) - z(i,j,k)
                dx(i,j,k)  = lx
                dy(i,j,k)  = ly
                dz(i,j,k)  = lz
                dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
                dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
                dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
            end do
        end do
    end do
end subroutine calc_serial
```

Последовательный алгоритм



```
!!$omp do перед внешним циклом k (непрерывный блок памяти обрабатывает каждая нить)
subroutine calc_omp_outer(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
    use omp_lib
    implicit none
    integer, intent(in) :: n, m
    real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
    real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    integer :: i, j, k
    real(8) :: lx, ly, lz

    !$omp parallel do schedule(runtime) private(i, j, k, lx, ly, lz)
    do k = 1, m
        do j = 1, m
            do i = 1, m
                lx = x(i+1,j,k) - x(i,j,k)
                ly = y(i,j+1,k) - y(i,j,k)
                lz = z(i,j,k+1) - z(i,j,k)
                dx(i,j,k)  = lx
                dy(i,j,k)  = ly
                dz(i,j,k)  = lz
                dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
                dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
                dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
            end do
        end do
    end do
    !$omp end parallel do
end subroutine calc_omp_outer
```

Параллельный цикл по внешнему индексу

1. *Последовательный алгоритм (SERIAL)*

Выполняется тройной цикл с оптимальным для Fortran порядком индексов k j i. Расчет идет строго на одном ядре процессора, директивы OpenMP не применяются.

2. *Параллельный цикл по внешнему индексу (omp_outer)*

Директива !$omp parallel do располагается перед внешним циклом по k, что позволяет разделить трехмерную область на непрерывные блоки слоев: каждый поток обрабатывает свой сплошной участок памяти, что минимизирует промахи кэша. Опция schedule (runtime) позволяет менять режим распределения итераций извне через переменную окружения OMP SCHEDULE.

3. *Варианты с переносом директивы и объединением циклов (omp_mid, omp_inner, collapse2, collapse3)*

В вариантах omp_mid и omp_inner (см.приложение) параллельная область !$omp parallel открывается один раз снаружи, а рабочая директива !$omp do ставится перед средним (j) или внутренним (i) циклом. В конце каждого внутреннего цикла потоки останавливаются на неявном барьере синхронизации.

В вариантах collapse2 и collapse3 (см.приложение) к директиве добавляется атрибут collapse (2) или collapse (3), который склеивает два или три вложенных цикла в одно длинное пространство итераций перед нарезкой между нитями.

4. *Параллельные секции по направлениям (SECTIONS)*

Конструкция !$omp parallel sections (см.приложение) делит вычисления на три независимые секции (!$omp section). Каждая секция берет на себя расчет расстояний только вдоль одного координатного направления и смежной с ним грани, после чего они выполняются параллельно на разных ядрах процессора.

6. Результаты проверки правильности выполнения программы и визуализация расчетной сетки

Для контроля корректности вычислений и проверки отсутствия гонки данных в параллельных версиях программы была введена контрольная сумма S по всем шести вычисленным трехмерным массивам расстояний для всех ячеек расчетной сетки:

$$S = \sum_{k=1}^{N-1} \sum_{j=1}^{N-1} \sum_{i=1}^{N-1} (d_{i,j,k}^{(x)} + d_{i,j,k}^{(y)} + d_{i,j,k}^{(z)} + d_{i,j,k}^{(xy)} + d_{i,j,k}^{(xz)} + d_{i,j,k}^{(yz)})$$

На расчетной сетке размером N = 300 ($299^3 = 26730899$ ячеек) последовательный алгоритм SERIAL и все разработанные параллельные модификации (omp_outer, omp_mid, omp_inner, collapse2, collapse3, SECTIONS) при различном числе потоков (от 1 до 8) выдали одинаковый результат:

S = 458330,299130

Незначительное расхождение в двенадцатом знаке на 8 потоках 458330,299142 объясняется естественной погрешностью округления при сложении миллионов чисел с плавающей точкой в разном порядке при параллельной редукции. Полное совпадение значений до десятого знака доказывает, что в программе отсутствует гонка данных, а все нити корректно записывают результаты в свои области памяти.

Для визуальной проверки правильности построения сетки и расчета ее геометрических характеристик был выполнен запуск программы на сетке размером N = 25 с выгрузкой центрального двумерного среза XOY при k = 12. С помощью Python и библиотеки matplotlib построены графики исходной декартовой сетки и полей вычисленных расстояний (см. рис. 2).

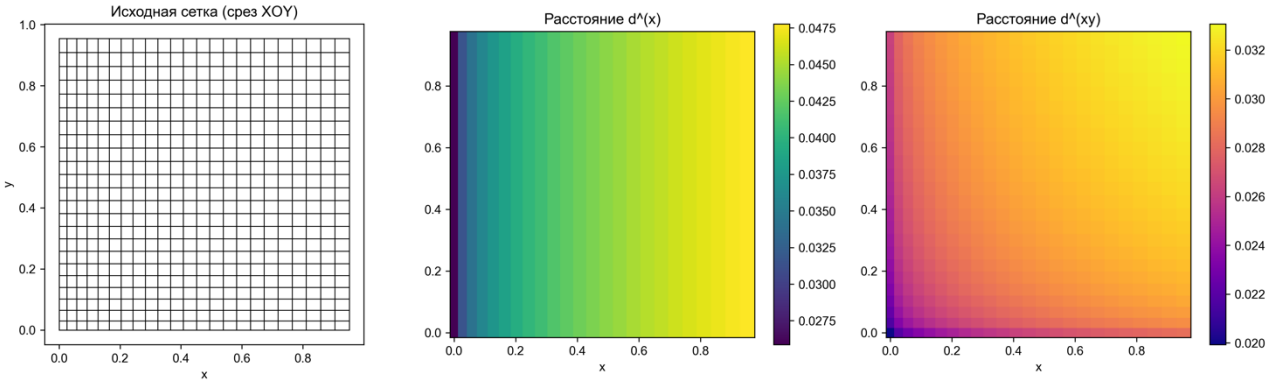


Рисунок 2. Исходная расчетная сетка (срез XOY) и поля расстояний между противоположными $d^{(x)}$ и смежными $d^{(xy)}$ гранями

На рисунке наглядно видно заданное сгущение узлов расчетной сетки вблизи начала координат и плавное увеличение расстояний между центрами противоположных и смежных граней по мере роста координат x и y, что полностью соответствует математическим формулам постановки задачи.

7. Исследование эффективности распараллеливания вложенных циклов

Все эксперименты проводились на фиксированном числе потоков p = 4 (производительные ядра процессора) для расчетной сетки размером N = 300 при порядке вложенности циклов kji.

Влияние местоположения директивы !$omp do

Было проверено, как зависит время работы программы с флагом оптимизации -O2 от того, перед каким из трех вложенных циклов стоит директива распараллеливания (см. таблицу 2 и рис. 3).

Режим	**Расположение директивы**	**Время выполнения (сек)**	**Ускорение относительно SERIAL = 0,1426 сек**
omp_outer	Перед внешним циклом (k)	0,081234	в 1,76 раза
omp_mid	Перед средним циклом (j)	0,100278	в 1,42 раза
omp_inner	Перед внутренним циклом (i)	1,794788	замедление в 12,6 раза

Таблица 2. Влияние расположения директивы !$omp do на время выполнения (4 потока, флаг -O2)

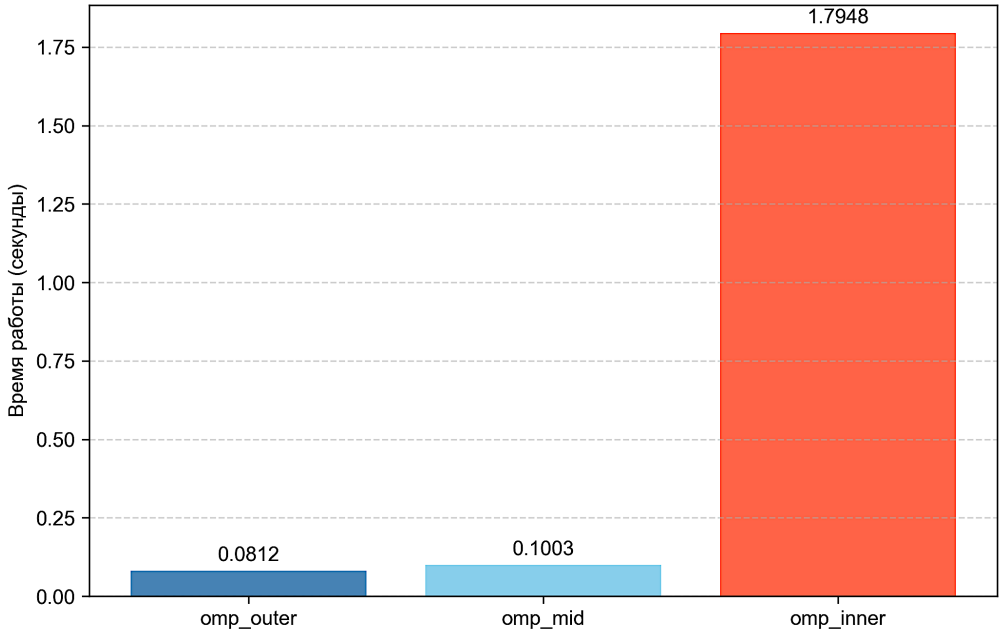


Рисунок 3. Влияние местоположения директивы !$omp do

Лучший результат = 0,0812 сек получен при распараллеливании внешнего цикла omp_outer. В этом случае потоки синхронизируются всего один раз в конце всей работы, а каждая нить получает непрерывный блок слоев и эффективно использует кэш процессора.

Перенос директивы на средний цикл omp_mid замедлил программу до 0,1003 сек, так как потокам пришлось проходить через барьер синхронизации 299 раз (на каждом шаге внешнего цикла k).

Размещение директивы перед внутренним циклом omp_inner вызвало снижение скорости до 1,7948 сек. В этом режиме барьер синхронизации срабатывал 299 × 299 = 89401 раз, а накладные расходы на управление нитями многократно превысили полезную работу над коротким циклом из 299 элементов.

Влияние способа распределения итераций (OMP_SCHEDULE)

Для варианта omp_outer с добавленной опцией schedule(runtime) было исследовано влияние переменной окружения OMP_SCHEDULE (режимы static с разными размерами блоков, dynamic и guided) при флаге -O2 (см. таблицу 3 и рис. 4).

OMP_SCHEDULE	Время выполнения, сек	Описание
Static	0,065268	Деление на 4 крупных непрерывных блока (лучший результат)
Static 20	0,072181	Чередование блоков по 20 слоев
Static 5	0,073547	Чередование блоков по 5 слоев
Static 1	0,078430	Циклическая нарезка по 1 слою
guided	0,078940	Динамическое уменьшение размера порций
dynamic	0,080490	Динамическая выдача по 1 слою через очередь
static,50	0,081072	Неравномерное распределение 299 слоев блоками по 50

Таблица 3. Влияние режимов планировщика OMP_SCHEDULE на время работы (4 потока, -O2)

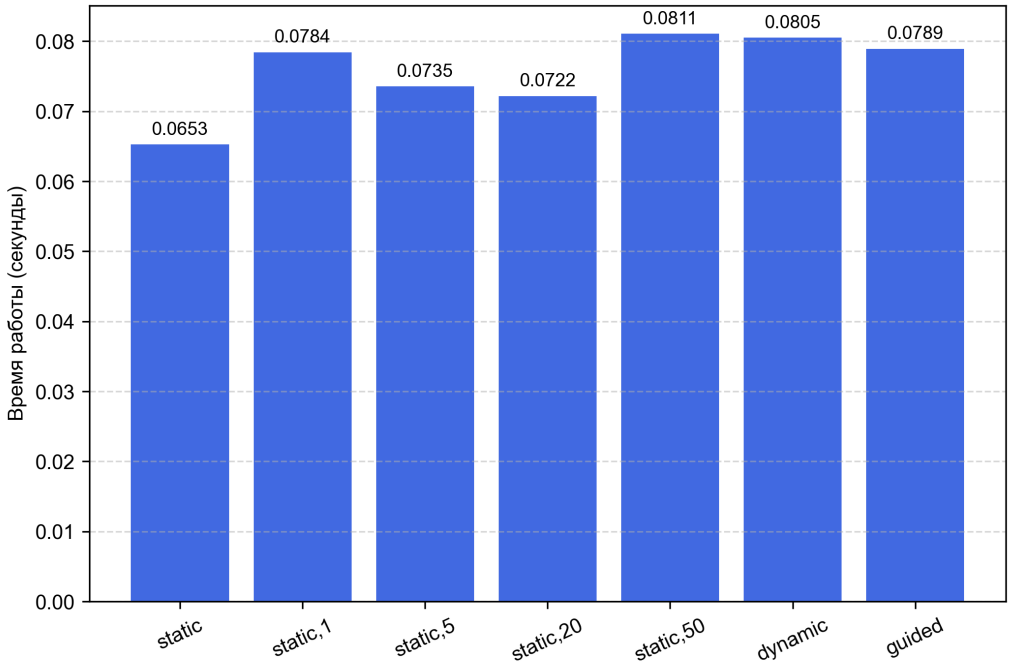


Рисунок 4. Влияние режимов планировщика OMP_SCHEDULE на время работы (4 потока, -O2)

Так как вычислительная сложность расчета для каждой ячейки сетки одинакова, лучше всего сработало статическое распределение по умолчанию (static 0,0653 сек). Оно заранее делит 299 слоев на 4 равных непрерывных куска без обращения к диспетчеру во время счета. Режимы dynamic (0,0805 сек) и guided (0,0789 сек) работали медленнее из-за лишних затрат времени на синхронизацию очереди задач.

Исследование атрибута collapse(m) при -O0 и -O2

Была изучена эффективность склеивания m вложенных циклов (collapse(1), collapse(2), collapse(3)) в одно пространство итераций при статическом распределении с размером блока 1, 10 и 100 в режимах без оптимизации (-O0) и с оптимизацией (-O2) (см. таблицу 4 и рис. 5).

Размер блока	collapse(m)	Без оптимизации (-O0)	С оптимизацией (-O2)
1	collapse(1) (omp_outer)	0,136671	0,085519
1	collapse(2)	0,258713	0,169278
1	collapse(3)	0,377984	0,338609
10	collapse(1) (omp_outer)	0,124951	0,082353
10	collapse(2)	0,128667	0,085145
10	collapse(3)	0,254507	0,220719
100	collapse(1) (omp_outer)	0,140008	0,075939
100	collapse(2)	0,122523	0,076258
100	collapse(3)	0,235862	0,190457

Таблица 4. Влияние глубины объединения циклов collapse(m) на время работы (4 потока, сек)

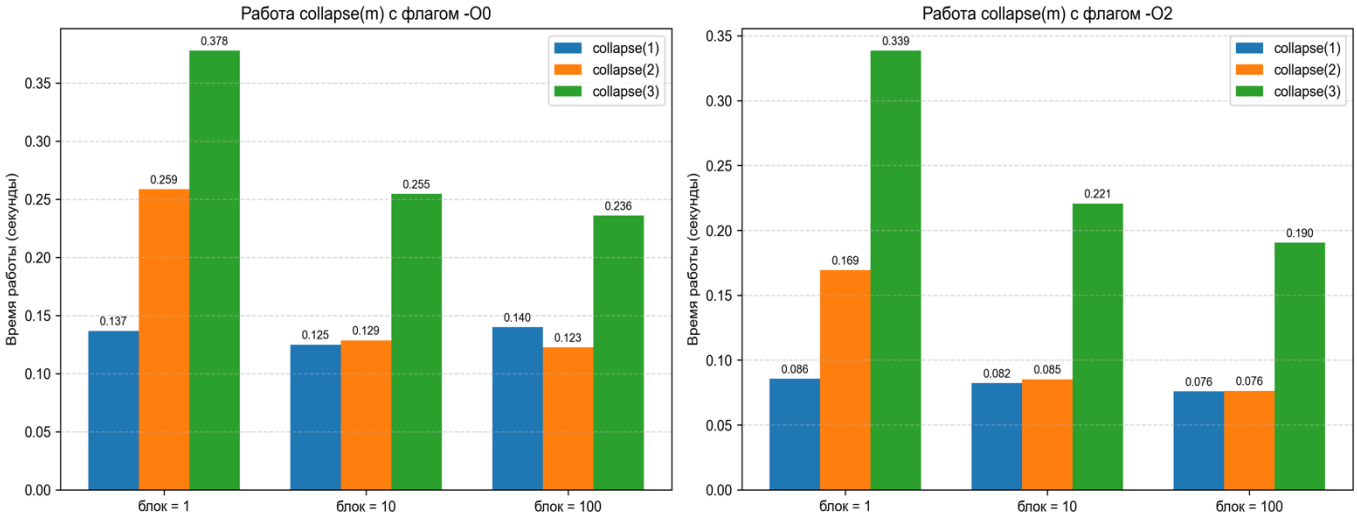


Рисунок 5. Влияние глубины объединения циклов collapse(m) на время работы (4 потока, сек)

Объединение двух внешних циклов (collapse(2)) при крупном размере блока 100 показывает скорость на уровне collapse(1) (0,0763 сек против 0,0759 сек при -O2), так как внутренний цикл по i остается целым и продолжает векторно обрабатывать данные подряд из кэша.

Объединение всех трех циклов (collapse(3)) во всех тестах привело к резкому замедлению (до 0,1905 - 0,3386 сек при -O2). При m = 3 планировщик начинает резать единый трехмерный цикл на мелкие кусочки отдельных ячеек. Это разрушает непрерывный обход по индексу I и добавляет накладные расходы на вычисление трехмерных координат из сквозного счетчика.

8. Исследование масштабируемости и сопоставительное сравнение параллельных подходов

Было проведено сравнительное исследование эффективности двух параллельных подходов при изменении числа вычислительных нитей p в диапазоне от 1 до 8 (с флагом оптимизации -O2 на сетке размером N = 300):

1. *Распараллеливание вложенных циклов* (omp_outer) наилучшая по результатам предыдущих исследований когфигурация с директивой !$omp parallel do перед внешним циклом k и крупноблочным статическим распределением OMP_SCHEDULE = static.
2. *Распараллеливание по независимым секциям* (SECTIONS) производит разделение вычислений на три параллельные секции (!$omp sections) по координатным направлениям X, Y, Z.

В качестве базового времени последовательного алгоритма для расчета ускорения и эффективности использовалось время выполнения однопоточной версии SERIAL, составившее 0,157230 секунды.

Результаты приведены в таблице 5 и на рисунке 6:

Число потоков p	**Время omp_outer (сек)**	**Ускорение Sp (do)**	**Эффективность Ep (do)**	**Время SECTIONS (сек)**	**Ускорение Sp (sections)**	**Эффективность Ep (sections)**
1	0,149947	1,05	1,05 (105%)	0,199812	0,79	0,79 (79%)
2	0,106835	1,47	0,74 (74%)	0,155674	1,01	0,50 (50%)
3	0,087801	1,79	0,60 (60%)	0,083110	1,89	0,63 (63%)
4	0,072288	2,18	0,54 (54%)	0,131872	1,19	0,30 (30%)
5	0,059451	2,64	0,53 (53%)	0,084228	1,87	0,37 (37%)
6	0,069162	2,27	0,38 (38%)	0,077857	2,02	0,34 (34%)
7	0,055734	2,82	0,40 (40%)	0,077609	2,03	0,29 (29%)
8	0,052998	2,97	0,37 (37%)	0,077643	2,03	0,25 (25%)

Таблица 5. Сравнение времени выполнения, ускорения и эффективности для !$omp do и !$omp sections (N = 300, -O2)

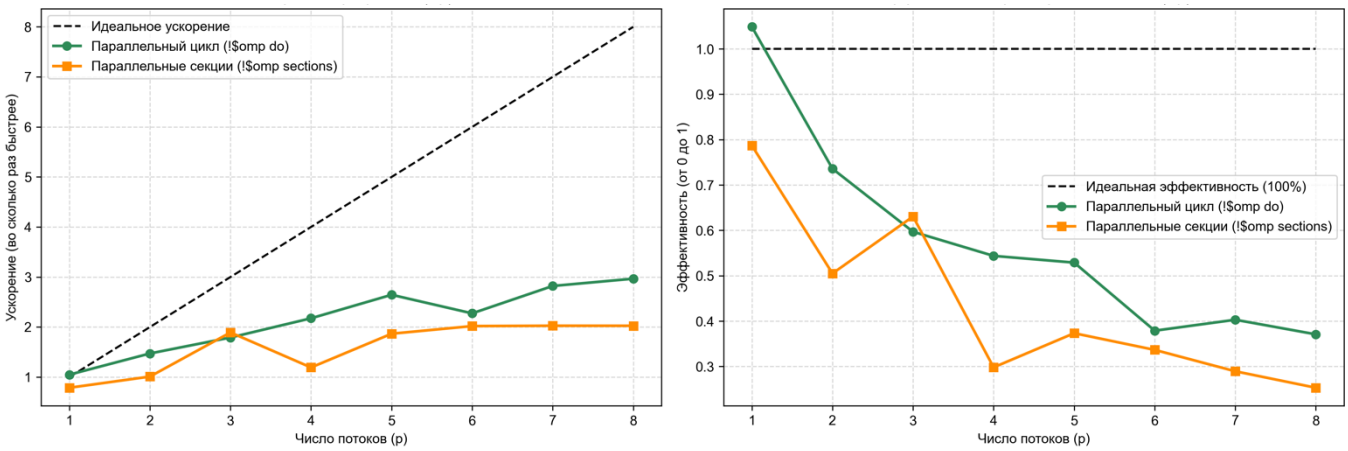


Рисунок 5. Графики ускорения Sp и эффективности распараллеливания Ep в зависимости от числа потоков p для параллельного цикла (!$omp do) и параллельных секций (!$omp sections)

Параллельные секции SECTIONS упираются в предел на 3 потоках. В коде всего три секции (по одной на каждую ось X, Y, Z). На 3 потоках каждое ядро берет ровно одну секцию — это лучший момент (0,083 сек, ускорение 1,89). На 4-8 потоках лишние ядра простаивают, поэтому время больше не падает (держится около 0,078 сек).

Параллельный цикл omp_outer ускоряется до всех 8 ядер. Цикл равномерно делится между любым числом потоков. Время падает от 0,150 сек на 1 потоке до 0,072 сек на 4 потоках и доходит до 0,053 сек на 8 потоках (максимальное ускорение 2,97).

Ускорение на 8 потоках не равно 8, что связано с тем, что у процессора M1 только 4 мощных ядра, а остальные 4 - слабые энергосберегающие. Также программа гоняет в памяти сразу 9 больших трехмерных массивов, и ядра начинают ждать оперативную память.

Выводы

В ходе работы были изучены методы распараллеливания вложенных циклов в технологии OpenMP на примере вычисления расстояний между геометрическими центрами граней ячеек трехмерной декартовой расчетной сетки размером 300 × 300 × 300 узлов.

Эксперименты подтвердили, что в языке Fortran с его размещением массивов по столбцам оптимальным является порядок вложенности циклов kji. Переход к обратному порядку ijk вызывает промахи мимо кэша процессора и замедляет программу более чем в 9. Включение оптимизации компилятора -O2 ускорило последовательный расчет в 2,7 раза за счет векторизации вычислений.

Все разработанные параллельные модификации выдали идентичную с последовательной версией контрольную сумму = 458330,299130, что доказывает отсутствие гонки данных при одновременной работе потоков с трехмерными массивами.

Наибольшую производительность обеспечивает размещение директивы !$omp do перед внешним циклом (omp_outer, 0,081 сек) при статическом распределении итераций по умолчанию (static, 0,065 сек). Это позволяет нарезать сетку на крупные непрерывные блоки памяти для каждой нити и свести количество барьеров синхронизации к одному. Перенос директивы на внутренний цикл (omp_inner) из-за 89401 барьера замедляет программу до 1,795 секунды.

Объединение двух внешних циклов (collapse(2)) при крупном блоке 100 работает на уровне collapse(1) (0,076 сек), тогда как склеивание всех трех циклов (collapse(3)) разрушает непрерывный доступ по внутреннему индексу i и замедляет вычисления более чем в 2,5 раза (до 0,190 секунды).

Подход на основе параллельных циклов (!$omp do) значительно превосходит метод параллельных секций (!$omp sections) при числе потоков p > 3. Секции ограничены тремя координатными направлениями и перестают масштабироваться после 3 нитей минимальное время 0,078 сек, ускорение 2,03, в то время как параллельный цикл продолжает ускоряться вплоть до всех 8 ядер процессора M1, достигая времени 0,053 секунды и максимального ускорения 2,97.

Приложение (код программы)

```
module variants_lib
   implicit none

   integer, parameter :: vUNKNOWN    = 0 !метод отслеживания ошибок
   integer, parameter :: vSERIAL     = 1 !последовательный kji
   integer, parameter :: vSERIAL_IJK = 2 !последовательный ijk
   integer, parameter :: vSERIAL_JKI = 3 !последовательный jki
   integer, parameter :: vOMP_OUTER  = 4 !!$omp do на внешнем цикле k + schedule(runtime)
   integer, parameter :: vOMP_MID    = 5 !!$omp do на среднем цикле j
   integer, parameter :: vOMP_INNER  = 6 !!$omp do на внутреннем цикле i
   integer, parameter :: vCOLLAPSE2  = 7 !!$omp do collapse(2) schedule(runtime)
   integer, parameter :: vCOLLAPSE3  = 8 !!$omp do collapse(3) schedule(runtime)
   integer, parameter :: vSECTIONS   = 9 !!$omp sections по направлениям x, y и z

contains

   !превращает константы vSERIAL, vSECTIONS,... в текстовое слово
   function var2str(var) result(str)
      integer, intent(in) :: var
      character(len=:), allocatable :: str
      select case (var)
         case (vSERIAL);     str = "SERIAL"
         case (vSERIAL_IJK); str = "serial_ijk"
         case (vSERIAL_JKI); str = "serial_jki"
         case (vOMP_OUTER);  str = "omp_outer"
         case (vOMP_MID);    str = "omp_mid"
         case (vOMP_INNER);  str = "omp_inner"
         case (vCOLLAPSE2);  str = "collapse2"
         case (vCOLLAPSE3);  str = "collapse3"
         case (vSECTIONS);   str = "SECTIONS"
         case default;       str = "UNKNOWN"
      end select
   end function var2str

   !аналогично в обратную сторону
   function str2var(str) result(var)
      character(len=*), intent(in) :: str
      integer :: var
      select case (str)
         case ("SERIAL");     var = vSERIAL
         case ("serial_ijk"); var = vSERIAL_IJK
```

```
        case ("serial_jki"); var = vSERIAL_JKI
        case ("omp_outer");  var = vOMP_OUTER
        case ("omp_mid");    var = vOMP_MID
        case ("omp_inner");  var = vOMP_INNER
        case ("collapse2");  var = vCOLLAPSE2
        case ("collapse3");  var = vCOLLAPSE3
        case ("SECTIONS");   var = vSECTIONS
        case default;        var = vUNKNOWN
      end select
   end function str2var

end module variants_lib

!начало основной программы
program main
   use variants_lib
   use omp_lib, only: omp_get_wtime, omp_get_max_threads
   implicit none

   integer, parameter :: Action_RUN = 0, Action_ERROR = 1 !флаги проверки правильности
программы

   type :: Options !структура настройки код выбранного метода (variant) и размер сетки N (ndiv)
      integer :: variant = vUNKNOWN
      integer(8) :: ndiv = 0 !число узлов сетки N по каждому направлению
   end type Options

   type ParseResult !структура результата разбора аргументов командной строки
      integer :: action = Action_RUN
      type(Options) :: options
      character(len=:), allocatable :: error
      character(len=:), allocatable :: argument
   end type ParseResult

   type(ParseResult) :: parsed !итог проверки аргументов терминала
   character(len=256) :: prog  !строка для хранения имени запущенной программы
   character(len=32) :: Sum_str, Time_str
   integer :: n, m, i, j, k, k_mid, unit_id
   real(8) :: total_sum, exec_time

   !динамические трехмерные массивы узлов сетки N x N x N и расстояний (N-1) x (N-1) x (N-1)
   real(8), allocatable :: x(:,:,:), y(:,:,:), z(:,:,:)
   real(8), allocatable :: dx(:,:,:), dy(:,:,:), dz(:,:,:), dxy(:,:,:), dxz(:,:,:), dyz(:,:,:)

   !интерфейсы всех вариантов вычислений
   interface
      subroutine calc_serial(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
         integer, intent(in) :: n, m
         real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
         real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
      end subroutine
      subroutine calc_serial_ijk(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
         integer, intent(in) :: n, m
         real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
         real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
      end subroutine
      subroutine calc_serial_jki(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
```

```
        integer, intent(in) :: n, m
        real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
        real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    end subroutine
    subroutine calc_omp_outer(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
        integer, intent(in) :: n, m
        real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
        real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    end subroutine
    subroutine calc_omp_mid(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
        integer, intent(in) :: n, m
        real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
        real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    end subroutine
    subroutine calc_omp_inner(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
        integer, intent(in) :: n, m
        real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
        real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    end subroutine
    subroutine calc_collapse2(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
        integer, intent(in) :: n, m
        real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
        real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    end subroutine
    subroutine calc_collapse3(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
        integer, intent(in) :: n, m
        real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
        real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    end subroutine
    subroutine calc_sections(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
        integer, intent(in) :: n, m
        real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
        real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    end subroutine
end interface

!вывод в консоль ошибок разбора аргументов
call get_command_argument(0, prog)
parsed = parse_input_arguments()

if (parsed%action == Action_ERROR) then
    call report_error(trim(prog), parsed%error, parsed%argument)
    stop 1
end if

n = int(parsed%options%ndiv)
if (n < 2) then
    call report_error(trim(prog), "Grid size N must be >= 2", "")
    stop 1
end if
m = n - 1
total_sum = 0.0d0

allocate(x(n,n,n), y(n,n,n), z(n,n,n))
allocate(dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m))

!инициализация исходной декартовой сетки
```

```
!$omp parallel do collapse(2) private(i, j, k)
do k = 1, n
   do j = 1, n
      do i = 1, n
         x(i, j, k) = (real(i - 1, 8) / real(m, 8)) ** 1.15d0
         y(i, j, k) = (real(j - 1, 8) / real(m, 8)) ** 1.10d0
         z(i, j, k) = (real(k - 1, 8) / real(m, 8)) ** 1.05d0
      end do
   end do
end do
!$omp end parallel do

exec_time = omp_get_wtime() !запуск секундомера
select case (parsed%options%variant) !переключатель кода метода
   case (vSERIAL);     call calc_serial(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vSERIAL_IJK); call calc_serial_ijk(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vSERIAL_JKI); call calc_serial_jki(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vOMP_OUTER);  call calc_omp_outer(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vOMP_MID);    call calc_omp_mid(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vOMP_INNER);  call calc_omp_inner(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vCOLLAPSE2);  call calc_collapse2(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vCOLLAPSE3);  call calc_collapse3(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case (vSECTIONS);   call calc_sections(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   case default; continue
end select
exec_time = omp_get_wtime() - exec_time !остановка секундомера

!контрольная сумма для проверки правильности расчета
!$omp parallel do collapse(2) reduction(+:total_sum) private(i, j, k)
do k = 1, m
   do j = 1, m
      do i = 1, m
         total_sum = total_sum + dx(i,j,k) + dy(i,j,k) + dz(i,j,k) &
                     + dxy(i,j,k) + dxz(i,j,k) + dyz(i,j,k)
      end do
   end do
end do
!$omp end parallel do

!вывод результатов
write(Sum_str, '(F20.6)') total_sum
write(Time_str, '(F20.9)') exec_time
write (*, '(A,",",I0,",",I0,",",A,",",A)') &
   var2str(parsed%options%variant), omp_get_max_threads(), &
   n, trim(adjustl(Sum_str)), trim(adjustl(Time_str))

!сохранение среза сетки и расстояний в файл для визуализации при N <= 50
if (n <= 50) then
   k_mid = max(1, m / 2)
   open(newunit=unit_id, file="grid_slice.csv", status="replace")
   write(unit_id, '(A)') "i,j,x,y,dx,dy,dxy"
   do j = 1, m
      do i = 1, m
         write(unit_id, '(I0,",",I0,5(",",F12.6))') i, j, &
            x(i,j,k_mid), y(i,j,k_mid), dx(i,j,k_mid), dy(i,j,k_mid), dxy(i,j,k_mid)
      end do
   end do
```

```
      close(unit_id)
   end if

   deallocate(x, y, z, dx, dy, dz, dxy, dxz, dyz)

contains

!защитный механизм который превращает аргумент терминала в число
   subroutine parse_positive_int(token, val, ok)
      character(len=*), intent(in) :: token
      integer(8), intent(inout) :: val
      logical, intent(inout) :: ok
      integer :: i, digit

      ok = .false.
      val = int(0, 8)
      if (len(token) == 0) return

      do i = 1, len_trim(token)
         if (token(i:i) < '0' .or. token(i:i) > '9') return
         digit = iachar(token(i:i)) - iachar('0')
         val = val * int(10, 8) + int(digit, 8) !сбор из цифр числа
      end do

      ok = (val > int(0, 8))
   end subroutine parse_positive_int

!запись информации об ошибке в res
   subroutine set_failure(res, message, argument)
      type(ParseResult), intent(inout) :: res
      character(len=*), intent(in) :: message
      character(len=*), intent(in) :: argument

      res%action = Action_Error
      res%error = message
      res%argument = argument
   end subroutine set_failure

!разбор аргументов командной строки и возврат структуры с результатом парсинга
   function parse_input_arguments() result(result_)
      type(ParseResult) :: result_
      integer :: i, argc, positional_count, arg_len
      character(len=:), allocatable :: token
      logical :: ok

      result_%action = Action_Run
      result_%error = ""
      result_%argument = ""

      argc = command_argument_count()
      positional_count = 0

      do i = 1, argc
         if (allocated(token)) deallocate(token)
         call get_command_argument(i, length=arg_len)
         allocate(character(len=arg_len) :: token)
         call get_command_argument(i, value=token)
```

```
            !первый аргумент имя метода
            if (positional_count == 0) then
               positional_count = positional_count + 1
               result_%options%variant = str2var(token)
               if (result_%options%variant == vUNKNOWN) then
                  call set_failure(result_, "Invalid directive name", token)
                  return
               end if
               cycle

            !второй аргумент размер сетки N
            else if (positional_count == 1 .and. len(token) > 0) then
               positional_count = positional_count + 1
               call parse_positive_int(token, result_%options%ndiv, ok)
               if (.not. ok) then
                  call set_failure(result_, "Invalid grid size", token)
                  return
               end if
               cycle
            end if

            call set_failure(result_, "Unexpected argument", token)
            return
         end do

         if (positional_count < 2) then
            call set_failure(result_, "Missing required arguments. Expected: NAME N", "")
         end if
      end function parse_input_arguments

!вывод сообщения об ошибке, правила использования и список директив
      subroutine report_error(prog_name, message, argument)
         character(len=*), intent(in) :: prog_name
         character(len=*), intent(in) :: message
         character(len=*), intent(in) :: argument

         write (*, '(2A)', advance='no') "Error: ", message
         if (len(argument) > 0) then
            write (*, '(3A)', advance='no') " -> '", argument, "'"
         end if
         write (*, '(A)') new_line('a')
         write (*, '(A)') "Usage: " // prog_name // " <DIRECTIVE NAME> <GRID SIZE>"
         write (*, '(A)') "Available names:"
         write (*, '(A)') "  SERIAL serial_ijk serial_jki omp_outer omp_mid omp_inner collapse2 collapse3
SECTIONS"
      end subroutine report_error

end program main

!последовательный kji (непрерывный доступ к памяти в fortran)
subroutine calc_serial(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
      implicit none
      integer, intent(in) :: n, m
      real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
      real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
      integer :: i, j, k
```

```
    real(8) :: lx, ly, lz

    do k = 1, m
       do j = 1, m
          do i = 1, m
             lx = x(i+1,j,k) - x(i,j,k)
             ly = y(i,j+1,k) - y(i,j,k)
             lz = z(i,j,k+1) - z(i,j,k)
             dx(i,j,k)  = lx
             dy(i,j,k)  = ly
             dz(i,j,k)  = lz
             dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
             dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
             dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
          end do
       end do
    end do
end subroutine calc_serial

!последовательный ijk (скачки по памяти и промахи мимо кэша)
subroutine calc_serial_ijk(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
    implicit none
    integer, intent(in) :: n, m
    real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
    real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    integer :: i, j, k
    real(8) :: lx, ly, lz

    do i = 1, m
       do j = 1, m
          do k = 1, m
             lx = x(i+1,j,k) - x(i,j,k)
             ly = y(i,j+1,k) - y(i,j,k)
             lz = z(i,j,k+1) - z(i,j,k)
             dx(i,j,k)  = lx
             dy(i,j,k)  = ly
             dz(i,j,k)  = lz
             dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
             dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
             dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
          end do
       end do
    end do
end subroutine calc_serial_ijk

!последовательный jki
subroutine calc_serial_jki(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
    implicit none
    integer, intent(in) :: n, m
    real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
    real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    integer :: i, j, k
    real(8) :: lx, ly, lz

    do j = 1, m
       do k = 1, m
          do i = 1, m
```

```
            lx = x(i+1,j,k) - x(i,j,k)
            ly = y(i,j+1,k) - y(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dx(i,j,k)  = lx
            dy(i,j,k)  = ly
            dz(i,j,k)  = lz
            dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
            dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
            dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
         end do
      end do
   end do
end subroutine calc_serial_jki

!!$omp do перед внешним циклом k (непрерывный блок памяти обрабатывает каждая нить)
subroutine calc_omp_outer(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   use omp_lib
   implicit none
   integer, intent(in) :: n, m
   real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
   real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
   integer :: i, j, k
   real(8) :: lx, ly, lz

   !$omp parallel do schedule(runtime) private(i, j, k, lx, ly, lz)
   do k = 1, m
      do j = 1, m
         do i = 1, m
            lx = x(i+1,j,k) - x(i,j,k)
            ly = y(i,j+1,k) - y(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dx(i,j,k)  = lx
            dy(i,j,k)  = ly
            dz(i,j,k)  = lz
            dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
            dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
            dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
         end do
      end do
   end do
   !$omp end parallel do
end subroutine calc_omp_outer


!!$omp do перед средним циклом j
subroutine calc_omp_mid(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   use omp_lib
   implicit none
   integer, intent(in) :: n, m
   real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
   real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
   integer :: i, j, k
   real(8) :: lx, ly, lz

   !$omp parallel private(i, j, k, lx, ly, lz)
   do k = 1, m
      !$omp do
```

```
      do j = 1, m
         do i = 1, m
            lx = x(i+1,j,k) - x(i,j,k)
            ly = y(i,j+1,k) - y(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dx(i,j,k)  = lx
            dy(i,j,k)  = ly
            dz(i,j,k)  = lz
            dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
            dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
            dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
         end do
      end do
      !$omp end do
   end do
   !$omp end parallel
end subroutine calc_omp_mid


!!$omp do перед внутренним циклом i
subroutine calc_omp_inner(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   use omp_lib
   implicit none
   integer, intent(in) :: n, m
   real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
   real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
   integer :: i, j, k
   real(8) :: lx, ly, lz

   !$omp parallel private(i, j, k, lx, ly, lz)
   do k = 1, m
      do j = 1, m
         !$omp do
         do i = 1, m
            lx = x(i+1,j,k) - x(i,j,k)
            ly = y(i,j+1,k) - y(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dx(i,j,k)  = lx
            dy(i,j,k)  = ly
            dz(i,j,k)  = lz
            dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
            dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
            dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
         end do
         !$omp end do
      end do
   end do
   !$omp end parallel
end subroutine calc_omp_inner
!!$omp do с collapse(2) и schedule(runtime)
subroutine calc_collapse2(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   use omp_lib
   implicit none
   integer, intent(in) :: n, m
   real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
   real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
   integer :: i, j, k
```

```
   real(8) :: lx, ly, lz

   !$omp parallel do collapse(2) schedule(runtime) private(i, j, k, lx, ly, lz)
   do k = 1, m
      do j = 1, m
         do i = 1, m
            lx = x(i+1,j,k) - x(i,j,k)
            ly = y(i,j+1,k) - y(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dx(i,j,k)  = lx
            dy(i,j,k)  = ly
            dz(i,j,k)  = lz
            dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
            dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
            dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
         end do
      end do
   end do
   !$omp end parallel do
end subroutine calc_collapse2


!!$omp do c collapse(3) и schedule(runtime)
subroutine calc_collapse3(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   use omp_lib
   implicit none
   integer, intent(in) :: n, m
   real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
   real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
   integer :: i, j, k
   real(8) :: lx, ly, lz

   !$omp parallel do collapse(3) schedule(runtime) private(i, j, k, lx, ly, lz)
   do k = 1, m
      do j = 1, m
         do i = 1, m
            lx = x(i+1,j,k) - x(i,j,k)
            ly = y(i,j+1,k) - y(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dx(i,j,k)  = lx
            dy(i,j,k)  = ly
            dz(i,j,k)  = lz
            dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
            dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
            dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
         end do
      end do
   end do
   !$omp end parallel do
end subroutine calc_collapse3


!!$omp sections три независимые секции по координатным направлениям x, y, z
subroutine calc_sections(x, y, z, dx, dy, dz, dxy, dxz, dyz, n, m)
   use omp_lib
   implicit none
   integer, intent(in) :: n, m
```

```
    real(8), intent(in) :: x(n,n,n), y(n,n,n), z(n,n,n)
    real(8), intent(out) :: dx(m,m,m), dy(m,m,m), dz(m,m,m), dxy(m,m,m), dxz(m,m,m), dyz(m,m,m)
    integer :: i, j, k
    real(8) :: lx, ly, lz

    !$omp parallel sections private(i, j, k, lx, ly, lz)
      !$omp section
      do k = 1, m
        do j = 1, m
          do i = 1, m
            lx = x(i+1,j,k) - x(i,j,k)
            ly = y(i,j+1,k) - y(i,j,k)
            dx(i,j,k)  = lx
            dxy(i,j,k) = 0.5d0 * sqrt(lx*lx + ly*ly)
          end do
        end do
      end do

      !$omp section
      do k = 1, m
        do j = 1, m
          do i = 1, m
            ly = y(i,j+1,k) - y(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dy(i,j,k)  = ly
            dyz(i,j,k) = 0.5d0 * sqrt(ly*ly + lz*lz)
          end do
        end do
      end do

      !$omp section
      do k = 1, m
        do j = 1, m
          do i = 1, m
            lx = x(i+1,j,k) - x(i,j,k)
            lz = z(i,j,k+1) - z(i,j,k)
            dz(i,j,k)  = lz
            dxz(i,j,k) = 0.5d0 * sqrt(lx*lx + lz*lz)
          end do
        end do
      end do
    !$omp end parallel sections
  end subroutine calc_sections
```